

How to Undo Bad Kubernetes Deployments with Helm

Deploying to Kubernetes can be stressful, but Helm makes recovery incredibly easy. This guide teaches you how to roll back broken updates and keep your applications running smoothly without panicking.

What you'll learn:

- 1 Step 1: Check Your Release History
- 2 Step 2: Roll Back to Safety
- 3 Step 3: Verify the Fix
- 4 Step 4: Audit Your Settings

Aman Raj Singh

Cloud Operations Engineer @ iManage

linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

Step 1: Check Your Release History

Before fixing anything, you need to know what went wrong and when. Helm tracks every single change you make as a revision number. By listing the history, you can see a timeline of your deployments, their status, and which revision was the last stable one.

```
$helm history my-app
```

Cloud & DevOps Daily

Helm Rollback: Undo a Bad Release in Under 30 Seconds

Monday, October 05, 2026

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

Step 2: Roll Back to Safety

Once you identify a stable revision number from your history, you can tell Helm to immediately revert your live application back to that exact state. Helm will automatically update Kubernetes to terminate the broken pods and spin up the old, working ones.

```
$helm rollback my-app 2
```

Cloud & DevOps Daily

Helm Rollback: Undo a Bad Release in Under 30 Seconds

Monday, October 05, 2026

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

Step 3: Verify the Fix

After running a rollback, you must verify that your application successfully recovered. Checking the release status shows you if the rollback succeeded and prints the active configuration, giving you peace of mind that the site is healthy again.

```
$helm status my-app
```

Cloud & DevOps Daily

Helm Rollback: Undo a Bad Release in Under 30 Seconds

Monday, October 05, 2026

Swipe to tip 4 >> linkedin.com/in/mramanrajsingh

4

TIP 4 OF 4

Step 4: Audit Your Settings

To prevent the same mistake from happening again, you should inspect the values that were actually deployed. This command shows you the custom configuration settings applied to your release, helping you spot any typos or wrong environment variables.

```
$helm get values my-app
```

Cloud & DevOps Daily

Helm Rollback: Undo a Bad Release in Under 30 Seconds

Monday, October 05, 2026

See Key Takeaways on next page >> linkedin.com/in/mrmanrajsingh

Key Takeaways

- + Always run a dry-run before upgrading to catch configuration typos early.
- + Keep your revision history clean by setting a limit using the `--history-max` flag.
- + Name your Helm releases clearly so you can easily identify them in a shared cluster.
- + Treat your Helm value files as code and store them in a Git repository.
- + Check your application logs immediately after a rollback to ensure traffic is flowing.

Watch Out For

- ! Rolling back to a version that has incompatible database schema changes.
- ! Forgetting to specify the correct Kubernetes namespace when running Helm commands.
- ! Manually editing Kubernetes resources with `kubectl` instead of using Helm, which causes configuration drift.
- ! Assuming a rollback worked without checking 'helm status' and pod health.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mrmanrajsingh